Installationshandbuch
tails-pdp
ASBAC-Prototyp im Linux-Kernel

Christian Koch

Stand: 25. Oktober 2026

Inhaltsverzeichnis

1 Zweck und Geltungsbereich **1**

2 Voraussetzungen und Lieferumfang **1**
2.1 Linux-Zielsystem 1
2.2 Buildumgebung und Dateien 2

3 Installation in drei Arbeitsschritten **3**
3.1 Schritt 1: Quellstand und Werkzeuge vorbereiten 3
3.2 Schritt 2: Programme bauen 3
3.3 Schritt 3: Runtime starten und Bereitschaft prüfen 4

4 Funktionsprobe mit einer temporären Policy **5**

5 Geführte Demonstration **6**

6 Prüfkette, Fehlerdiagnose und Grenzen **6**

7 Dokumentausgabe und Nachweisstand **7**

1 Zweck und Geltungsbereich

Dieses Handbuch beschreibt den Build und die Inbetriebnahme des Prototyps auf einem vorbereiteten Linux-Zielsystem in drei Arbeitsschritten. Es bezieht sich auf die bereinigte Repository-Struktur mit den Einstiegsskripten unter `scripts/`. Der Prototyp kontrolliert neue Dateiöffnungen über den BPF-LSM-Hook `file_open` und bewertet bereits offene Dateideskriptoren im Userspace nach. Socket-Bind-Policies werden nicht unterstützt.

Das Zielsystem wird vor der Installation bereitgestellt. Die Einrichtung des Hypervisors und die Installation des Betriebssystems sind Voraussetzungen dieses Ablaufs. Die Referenzkonfiguration beschreibt eine dedizierte Test-VM; die Installation ist für eine solche Umgebung vorgesehen. Die nachfolgenden Befehle werden in Bash ausgeführt. Platzhalter wie `REPOSITORY-URL` und `ABGABE-TAG` sind durch die tatsächlich bereitgestellten Werte zu ersetzen.

2 Voraussetzungen und Lieferumfang

2.1 Linux-Zielsystem

Als Referenz dienen NixOS auf x86_64 und Linux 6.16.12. Die in der Evaluation verwendete VM besitzt vier virtuelle CPUs, 8192 MiB RAM und eine 64-GiB-Systemplatte. Diese Ausstattung ist eine Referenz, keine gemessene Mindestanforderung. Andere Kernel und Architekturen benötigen eine eigene Funktionsprüfung.

Benötigt werden:

- ein Kernel mit eBPF, BPF-LSM und Kernel-BTF;
- `bpf` in der aktiven LSM-Reihenfolge;
- lesbare BTF-Daten unter `/sys/kernel/btf/vmlinux`;
- eingehängte Dateisysteme `securityfs` unter `/sys/kernel/security` und `bpf` unter `/sys/fs/bpf`;
- Root-Zugriff über `sudo` für Laden, Attach, Map-Zugriff und `ptrace`-basierten FD-Entzug;
- Git, Nix mit `nix-shell`, Bash und die lokal vorbereiteten Buildwerkzeuge; für die geführte Demo außerdem `tmux`, Python 3, `flock`, `unshare` und `mount`. Ein Netzwerkzugang kann zur vorgelagerten Beschaffung der Werkzeuge nötig sein; der Cargo-Build verwendet die mitgelieferten Abhängigkeitsquellen.

Die Dateien `nix-config/configuration.nix` und `nix-config/hardware-configuration.nix` dokumentieren die Referenzinstallation. Die Hardwaredatei gehört zum konkreten Gast und darf auf einem anderen Rechner nicht ungeprüft dessen Hardwarekonfiguration ersetzen. Die enthaltenen Initialpasswörter `password` sind Demowerte; Zugangsdaten des bereitgestellten Zielsystems werden separat eingerichtet. Die benötigten Kerneloptionen und LSM-Einstellungen sind durch die Systemadministration vorab bereitzustellen.

Die aktive Umgebung wird geprüft mit:

```
uname -m
uname -r
cat /sys/kernel/security/lsm
test -r /sys/kernel/btf/vmlinux
findmnt -t bpf /sys/fs/bpf
sudo -v
```

In der LSM-Ausgabe muss `bpf` als Eintrag vorhanden sein. Im ausgewerteten Referenzlauf lautete die Reihenfolge `capability,landlock,yama,bpf`. Fehlende Kernelvoraussetzungen werden vor dem Build behoben. Falls nur das BPF-Dateisystem noch nicht eingehängt ist, kann die Administration es mit `sudo mount -t bpf bpf /sys/fs/bpf` einhängen.

2.2 Buildumgebung und Dateien

Die Entwicklungsumgebung wird über `nix-config/shell.nix` bereitgestellt: LLVM 21 einschließlich Laufzeitbibliothek, Rustup, Bash, Coreutils, GNU Grep, Procps, Python 3 und tmux. Die Dateiinhalte von `nix-config/rust-toolchain` und `nix-config/ebpf-toolchain` legen die Toolchains fest: aktuell `1.93.0` für Userspace und `nightly-2026-01-15` für eBPF. Benötigt werden außerdem Rustfmt und Clippy für die Userspace-Toolchain, `rust-src` für die eBPF-Toolchain sowie `bpf-linker` 0.9.15 mit LLVM-21-Unterstützung. Der aktuelle Aya-eBPF-Build benötigt weiterhin Nightly. Die Nix-Shell setzt `RUSTUP_AUTO_INSTALL=0` und installiert weder Toolchains noch den Linker beim Start nach.

`Cargo.lock` legt die Projektabhängigkeiten fest. Die Quellen unter `vendor/` umfassen Registry- und Git-Abhängigkeiten einschließlich Aya sowie die Registry-Pakete für den eBPF-Systembibliotheksbuild. Dessen Liste liegt unter `nix-config/ebpf-sysroot/`. Die Quellenersetzung in `.cargo/config.toml` ist bereits aktiviert; Cargo arbeitet standardmäßig offline. `scripts/check-vendor.py` prüft Paketdateien, Prüfsummen und auf Wunsch auch die Abhängigkeitsliste der installierten `rust-src`-Komponente.

Die Shell importiert <nixpkgs> ohne feste Revision. Compiler, Rust-Quellen, Linker, LLVM-Bibliotheken und benötigte Nix-Store-Pfade müssen vorher lokal vorhanden sein. Ein vollständiges Offline-Werkzeugpaket einschließlich Nixpkgs und Nix-Store- Abhängigkeiten ist derzeit nicht mitgeliefert. Das Cargo-Vendoring allein ersetzt keine vorbereitete System- und Werkzeugumgebung.

Der Quellumfang umfasst die acht `tails-pdp*`-Crates, Cargo-Manifeste, Lockdatei, `.cargo/config.toml`, Skripte, Konfigurationsbeispiele und Lizenzen. `tails-pdp-ebpf/src/vmlinux.rs` wird zum Build benötigt. Das eBPF-Objekt wird durch das Cargo-Buildskript erstellt und in das Hauptprogramm eingebettet; es muss zur Ausführung nicht als separate Datei installiert werden. Das Evaluationsarchiv liegt unter `eval-tails-pdp/`.

3 Installation in drei Arbeitsschritten

3.1 Schritt 1: Quellstand und Werkzeuge vorbereiten

Den bereitgestellten Abgabestand beziehen und eine Nix-Shell öffnen:

```
git clone REPOSITORY-URL tails-pdp
cd tails-pdp
git checkout ABGABE-TAG
nix-shell nix-config/shell.nix
```

Bei einer Übergabe als Quellarchiv wird dieses stattdessen entpackt und in sein Wurzelverzeichnis gewechselt. Die Toolchains, der Linker und die Nix-Abhängigkeiten müssen auf dem Zielsystem bereits bereitgestellt sein. Innerhalb der Nix-Shell prüfen:

```
rustup run "$(cat nix-config/rust-toolchain)" rustc --version
rustup run "$(cat nix-config/ebpf-toolchain)" rustc --version
setup-bpf-tools
bpf-linker -V
```

`setup-bpf-tools` prüft ausschließlich den vorhandenen Linker auf Version 0.9.15; die Funktion installiert nichts. Die reguläre Toolchain wird aus `nix-config/rust-toolchain` gelesen, die eBPF-Toolchain aus `nix-config/ebpf-toolchain`. Ein unversioniertes `stable` oder `nightly` ersetzt diese Festlegung nicht.

Falls die Toolchains bei der vorgelagerten Vorbereitung mit Netzwerkzugang noch beschafft werden müssen, können die festgelegten Versionen ausdrücklich installiert werden:

```
rustup toolchain install "$(cat nix-config/rust-toolchain)" --component rustfmt
    --component clippy
rustup toolchain install "$(cat nix-config/ebpf-toolchain)" --component rust-src
```

Die passende Linkerinstallation und alle Nix-Pakete sind ebenfalls vor dem Offline-Betrieb bereitzustellen. Die Shell stellt zusätzlich die Funktion `tails-pdp-admintool` bereit. Sie bestimmt die Repository-Wurzel aus der Nix-Datei und ruft das dort gebaute Admintool mit `sudo` auf. Die folgenden Beispiele verwenden den expliziten Binary-Aufruf.

3.2 Schritt 2: Programme bauen

Im Repository-Wurzelverzeichnis innerhalb der Nix-Shell ausführen:

```
bash scripts/demo/build.sh
test -x target/release/tails-pdp
test -x target/release/tails-pdp-admintool
```

Der gemeinsame Buildhelfer prüft die festgelegten Toolchains, `rust-src`, die Vendor-Prüfsummen und `bpf-linker` 0.9.15. Danach baut er Runtime und Admintool einschließlich des eingebetteten eBPF-Programms mit `cargo build -frozen`. `-frozen` verbindet eine unveränderte Abhängigkeitsauflösung mit dem Offline-Modus. Fehlende Quellen oder Werkzeuge führen zum Abbruch und werden nicht automatisch installiert.

Der entsprechende explizite Cargo-Aufruf lautet:

```
CARGO_NET_OFFLINE=true RUSTUP_AUTO_INSTALL=0 cargo build --frozen --release --bin
     tails-pdp --bin tails-pdp-admintool
```

Alternativ ist weiterhin ein ausdrücklicher Offline-Build möglich:

```
CARGO_NET_OFFLINE=true cargo build --locked --offline --release --bin tails-pdp
    --bin tails-pdp-admintool
```

`CARGO_NET_OFFLINE=true` gilt auch für verschachtelte Cargo-Prozesse des eBPF-Builds. Der Cargo-Direktaufruf ersetzt die zusätzlichen Prüfungen des Buildhelfers nicht. Toolchains und Systemwerkzeuge gehören nicht zu `vendor/`. Die unter `runtime/bin/` bereitgestellten Binaries besitzen einen Nix-spezifischen dynamischen Interpreter und benötigen dessen Laufzeitabhängigkeiten; sie sind kein allgemein auf beliebigen Linux-Systemen ausführbares Paket.

3.3 Schritt 3: Runtime starten und Bereitschaft prüfen

Vor dem Start sicherstellen, dass keine weitere Runtime aktiv ist:

```
pgrep -x tails-pdp
```

Eine vorhandene Instanz zuerst regulär beenden. Bei einem frischen Checkout sind keine Beispiel-Policies automatisch aktiviert. Verbliebene lokale `.policy`-Dateien unter `policies/` vorab prüfen. Dann aus der Repository-Wurzel starten:

```
sudo env RUST_LOG=info ./target/release/tails-pdp
```

Das Arbeitsverzeichnis ist Teil der Konfiguration: Die Runtime lädt Policies aus `policies/` und Attribute aus `attributes/`. Fehlende Verzeichnisse werden angelegt. Die Runtime bleibt im Vordergrund. Im Log müssen das Beobachten des Policyverzeichnisses und `Waiting for Ctrl-C...` erscheinen; Attach-Fehler dürfen nicht auftreten. Eine zweite

Shell kann den Prozess und die aktiven Daten prüfen:

```
pgrep -x tails-pdp
sudo ./target/release/tails-pdp-admintool show-active
```

Ein leeres Policy-Ergebnis ist beim Erststart zulässig. Ein bloß vorhandener Map-Pin beweist keine laufende Runtime. Mit Strg+C wird die Runtime regulär beendet. Gepinnte Maps können danach weiterhin vorhanden sein.

Alternativ bündelt `scripts/run.sh` den Admintool-Build und den Runtime-Start:

```
./scripts/run.sh
```

Das Skript bestimmt selbst die Repository-Wurzel und verwendet über die Cargo-Konfiguration `sudo -E` als Runner. Es aktualisiert den Checkout nicht.

4 Funktionsprobe mit einer temporären Policy

Die folgende Probe demonstriert die statische Dateikontrolle für `cat`. Sie wird bei laufender Runtime in einer zweiten Shell aus der Repository-Wurzel ausgeführt. Andere aktive Policies können das Ergebnis beeinflussen; für diese Probe einen sonst leeren Policystand verwenden.

```
printf 'demo\n' > /tmp/tails-pdp-install-demo.txt
cat > /tmp/tails-pdp-install-demo.policy <<'POLICY'
policy "installation demo"
deny
    action == "file_open";
    command == "cat";
    resource.path == "/tmp/tails-pdp-install-demo.txt";
POLICY
bash scripts/publish-config.sh /tmp/tails-pdp-install-demo.policy policies/
    install-demo.policy
sudo ./target/release/tails-pdp-admintool show-active
cat /tmp/tails-pdp-install-demo.txt
```

Die Veröffentlichung wird asynchron validiert und aktiviert. Erst nach Bestätigung der Aktivierung im Runtime-Log oder Admintool ist die neue Regel wirksam. `cat` muss dann mit einer verweigerten Dateiöffnung enden. Ein unmittelbar nach der Veröffentlichung noch erfolgreicher Aufruf kann vor der Aktivierung liegen und wird danach wiederholt.

Zum Aufräumen:

```
rm policies/install-demo.policy
```

Nach Bestätigung der neuen Policygeneration muss `cat` wieder erfolgreich sein. Anschließend:

```
rm /tmp/tails-pdp-install-demo.policy /tmp/tails-pdp-install-demo.txt
```

Änderungen an aktiven Policy- und Attributdateien stets über `scripts/publish-config.sh` veröffentlichen, statt sie während des Betriebs direkt zu überschreiben. Die Beispiel-Policies mit `/home/hntr/test.txt` benötigen eine passende Ressource und gegebenenfalls angepasste UID-/Attributdaten.

5 Geführte Demonstration

Auf dem vorbereiteten dedizierten x86_64-Linux-System kann die vollständige Terminal-Demo verwendet werden. Eine zuvor manuell gestartete Runtime beenden; Demo und Evaluationssuiten nicht gleichzeitig ausführen.

```
./scripts/demo.sh
./scripts/demo.sh --auto
./scripts/demo.sh --build
```

Ohne Option führt Enter zum nächsten Schritt. `-auto` führt die Schritte ohne Eingabepausen aus; `-build` erzwingt vorher einen neuen Release-Build. Fehlende Binaries lösen ebenfalls den Build über die Nix-Umgebung aus. Werkzeuge und Nix-Abhängigkeiten müssen lokal vorliegen; die Demo lädt sie nicht nach. Ein eingerichtetes System-`sudo` beziehungsweise Root-Rechte bleiben erforderlich.

Die Demo verwendet tmux und einen privaten Mount-Namespace mit eigenem bpffs. Vorhandene Host-Pins bleiben erhalten. Die Konfigurationsverzeichnisse und `demo/files/` werden für die Demo vorbereitet; vorhandene Inhalte werden gesichert und beim regulären Aufräumen wiederhergestellt. Logs liegen unter `demo/runs/`. Einzelheiten und Diagnoseoptionen beschreibt `scripts/demo/README.md`. Ein erfolgreicher Demolauf ist vom archivierten Evaluationslauf und dessen Messwerten getrennt zu behandeln.

6 Prüfkette, Fehlerdiagnose und Grenzen

Die vollständige Prüfung erfolgt ausschließlich auf dem dedizierten Linux-Zielsystem, nach Beenden der Demonstrationsruntime. Eine weiterhin wirksame Demo-Policy vorher entfernen.

```
./scripts/test.sh
```

Die Prüfkette umfasst Formatierung, Veröffentlichungstest, Demo-Syntax- und Regressionstests, Rust-Tests, Python-Messauswertung, Clippy, Release-Build, privilegierte Systemtests und eine automatisch geprüfte vollständige tmux-Demo. Sie erzeugt ein Ergebnisverzeichnis unter `test-results/`. Die privilegierten Tests verwalten temporär bekannte Maps

und isolierte Runtime-Fixtures. Vorhandene Pins und Prozesse können einen Teststart verhindern.

Bei unvereinbaren gepinnten Map-Layouts oder vor einem ausdrücklich vorgesehenen Testlauf die Runtime zunächst beenden und die bekannten Pins bereinigen:

```
sudo ./scripts/remove-maps.sh
```

Das Skript entfernt aktuelle Pins und historische Pins früherer Fassungen. Es beendet keine Runtime. Es darf deshalb nicht als laufender Konfigurationswechsel verwendet werden.

Buildfehler oder fehlender Linker
: Innerhalb der Nix-Shell bauen, `setup-bpf-tools` ausführen und Rust-Toolchains sowie `rust-src` prüfen. Den vollständigen Compilerfehler auswerten.

Fehlendes BTF oder fehlgeschlagener LSM-Attach
: Kernelvoraussetzungen, aktive LSM-Reihenfolge und BTF-Datei prüfen. MacOS ist kein Ausführungsziel für die Runtime.

Fehler beim Pinnen oder Öffnen von Maps
: BPF-Mount, Root-Berechtigungen und Map-Layout prüfen. Alte Pins nur nach Beenden der Runtime entfernen.

Policy oder Attribut wird nicht übernommen
: Arbeitsverzeichnis, Dateiendung und Runtime-Log prüfen. Ungültige Updates lassen die letzte gültige Generation bestehen; initial ungültige Daten können den Start verhindern. Veröffentlichungsbestätigung allein ist keine Aktivierungsbestätigung.

FD-Entzug oder privilegierter Test schlägt fehl
: Root-/ptrace-Berechtigungen und den konkreten Testreport prüfen. Die Userspace-Nachbewertung ist nicht atomar mit sämtlichen Dateioperationen; ein erfolgreicher Einzellauf beweist keine Race-Freiheit.

7 Dokumentausgabe und Nachweisstand

Die LaTeX-Datei ist eigenständig und benötigt keine eingebundenen Thesis-Dateien, Literaturdaten oder Abbildungen. Für den gemeinsamen Build liegen die Quellen unter `../thesis/`; benötigt werden LaTeX, `latexmk` und Biber. Aus der Repository-Wurzel ausführen:

```
bash scripts/build-thesis.sh
```

Die lokale Laufkonfiguration `thesis` ruft denselben Helfer auf. Er baut Thesis und beide Handbücher und kopiert erst nach erfolgreichem Build aller drei Dokumente die PDFs nach `thesis/final/`. Dieses Handbuch erhält dort den Namen `Christian_K`

och_Installationshandbuch_tails-pdp.pdf; die Thesis wird mit Namen und Arbeitstitel benannt. Die Handbuchquellen bleiben im vorgesehenen Thesis-/source-Paket. Eine separate Kompilierung dieses Handbuchs ist weiterhin möglich.

Die dokumentierten Programme, Skripte und Pfade wurden mit dem bereinigten Quellstand abgeglichen. Die vollständige Neuinstallation nach diesem Handbuch ist noch auf dem vorgesehenen Linux-Zielsystem zu prüfen. Das archivierte Evaluationsergebnis vom 10. Oktober 2026 zum Commit a9fd6c0 belegt den damaligen Quellstand; spätere Bereinigungen und dieses Handbuch sind damit nicht automatisch erneut evaluiert. Für die finale Übergabe sind der tatsächliche Abgabetag und die ausgelieferten Binärartefakte eindeutig zuzuordnen.